

Processes

Definition

A **process** is a program in execution. It is one of the most important concepts in operating systems. A process is not just the program code; it also includes memory, resources, and all information needed to run that program.

1. What a Process Contains

Main Components

Each process generally contains:

- an **address space**,
- the **executable program**,
- the program's **data**,
- the **stack**,
- **register values**,
- **open files**,
- **alarms and signals**,
- information about **related processes**.

Address Space

The **address space** is the range of memory locations a process can use. It contains the program code, data, and stack.

2. Process in a Multiprogramming System

Idea

In a multiprogramming system, several processes may exist at the same time. For example, a user may run a video editor, a web browser, and an email receiver simultaneously.

CPU Switching

The operating system does not run all processes at the exact same instant on one CPU. Instead, it switches the CPU from one process to another. This gives the appearance that many programs are running together.

3. Suspending and Restarting a Process

Saving Process State

When a process is temporarily stopped, it must later continue in exactly the same state. So the operating system must save all important information before suspension.

Saved Information

The following information may need to be saved:

- register contents,
- program counter,
- stack pointer,
- open file information,
- current file positions,
- other execution details.

Process Table

In many operating systems, information about each process is stored in a **process table**. It is usually an array of entries, with one entry for each active process.

Core Image

A suspended process consists of:

- its **address space** or **core image**,
- its **process table entry**.

4. Process Creation and Termination

Basic System Calls

Important process-management system calls deal with:

- creating a process,
- terminating a process,
- waiting for a child process,
- requesting or releasing memory,
- replacing a program with another one.

Example

A command interpreter or **shell** reads user commands. If the user asks to compile a program, the shell creates a new process to run the compiler. After finishing, that process terminates itself.

5. Parent and Child Processes

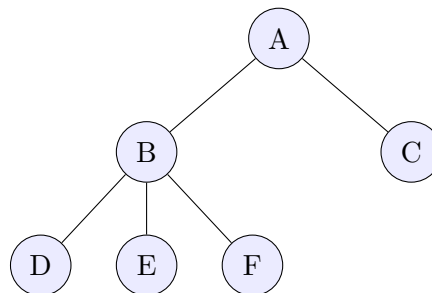
Relationship

A process can create another process. The creating process is called the **parent process**, and the new one is called the **child process**.

Process Tree

If child processes create more child processes, a **process tree** is formed.

5.1 Simple Process Tree



Explanation

Here:

- Process A creates B and C
- Process B creates D, E, and F

6. Interprocess Communication

Meaning

When related processes work together, they often need to exchange information and coordinate their activities. This is called **interprocess communication (IPC)**.

Purpose of IPC

IPC is needed for:

- data sharing,
- synchronization,
- cooperation between related processes.

7. Signals and Alarms

Signal

A **signal** is a notification sent to a process. It causes the process to temporarily stop its current activity and run a special signal-handling routine.

Alarm Signal

A process may request the operating system to send it an alarm after a certain number of seconds. This is useful when the process needs a timeout mechanism.

Example

If a process sends a message over a network and no reply arrives in time, an alarm signal may be used to trigger retransmission.

Important Note

Signals are the software equivalent of hardware interrupts. They may occur because of:

- timer expiration,
- illegal instruction,
- invalid memory address,
- other exceptional events.

8. User IDs and Group IDs

UID and GID

Each authorized user in a system is assigned a **UID** (User Identification). Users may also belong to groups, each identified by a **GID** (Group Identification).

Key Ideas

- Every process has the UID of the user who started it.
- A child process usually has the same UID as its parent.
- Group membership helps control access and permissions.

9. Superuser / Administrator

Special User

One special user has extra privileges:

- **Superuser** in UNIX
- **Administrator** in Windows

This user can override many protection rules.

Note

In large systems, only the system administrator should know the password for this powerful account.

10. Conclusion

Summary

A process is the basic unit of execution in an operating system. It includes the program, memory, resources, and control information needed for execution. Process creation, suspension, communication, and protection are central parts of operating system design.